UNIT -II

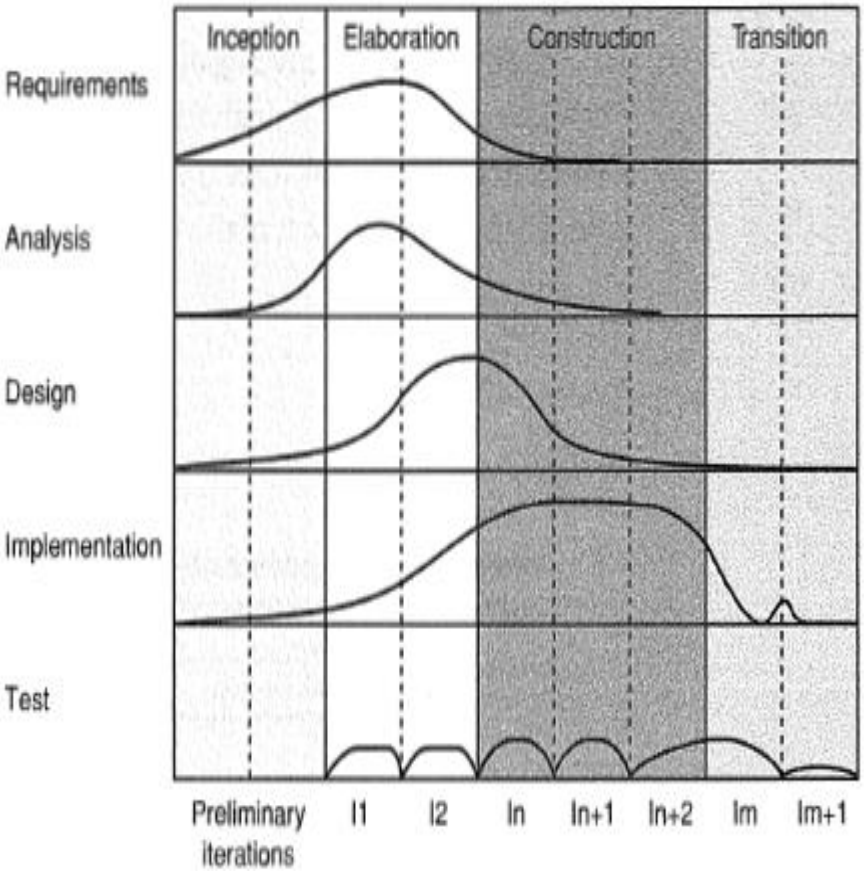
1. Analysis Work Flow:

The main work in analysis begins toward

- the end of the Inception phase and
- the main focus of the Elaboration phase, along with requirements.

The Main aim of analysis workflow is

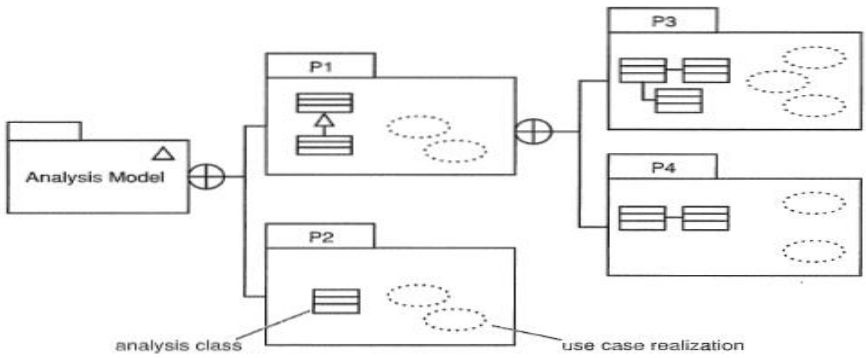
- To produce an analysis model.



A. Analysis artefacts -Metamodel

Two key artifacts are as follows

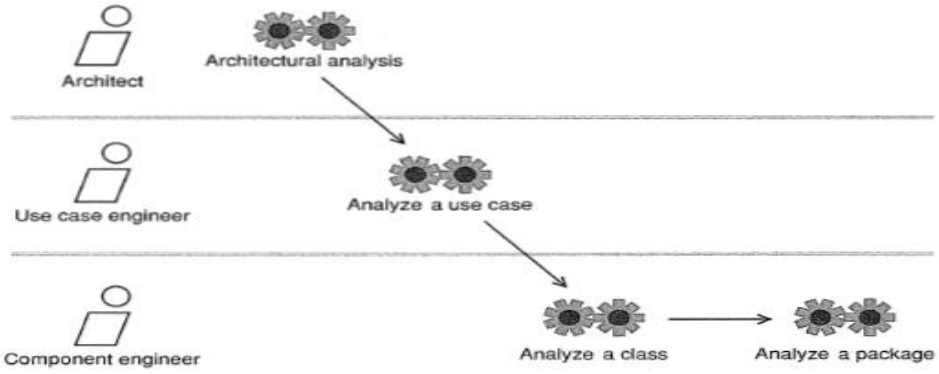
a. Analysis classes
b. use case realizations



B. *UP analysis workflow*

It has 4 stages. Those are as follows

- Architectural analysis;
- Analyze a use case;
- Analyze a class;
- Analyze a package

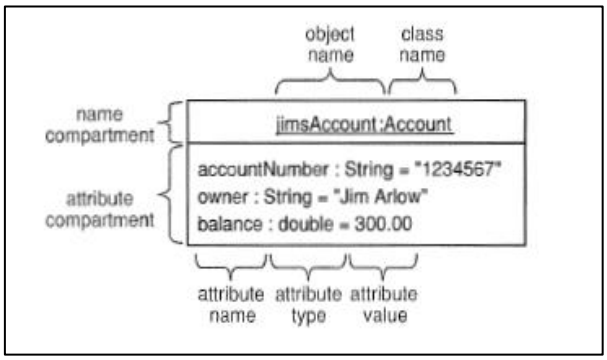


C. *Analysis model - rules of thumb:*

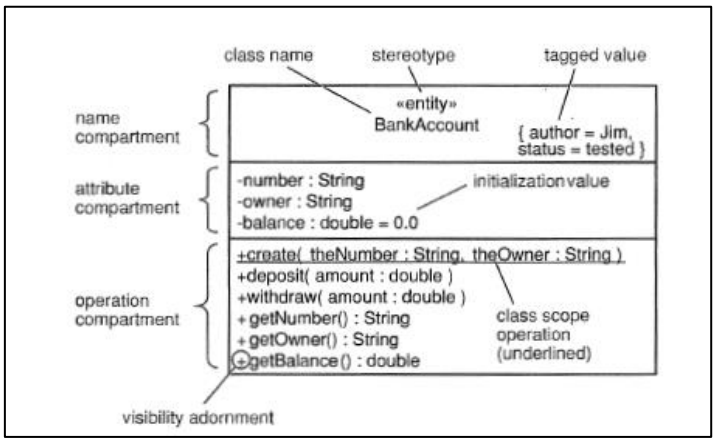
- 50 to 100 analysis classes in the analysis model.
- only include classes - the vocabulary of the problem domain related
- Do *not* make implementation decisions;
- Focus on classes and associations - minimize coupling;
- Use inheritance where there is a natural hierarchy of abstractions;
- keep it simple!

2. Objects and classes:

- ***Object Notation:***



- ***Class Notation:***



3. Finding Classes

- The way of finding the classes are as follows
    1. by using noun/verb analysis
    2. by using CRC analysis
    3. by using the RUP stereotypes
    4. from other sources

a. Noun/verb analysis:

a. Nouns and noun phrases in the text indicate classes or attributes of classes.
b. Verbs and verb phrases indicate responsibilities or operations of a class.

Procedure:

- The first step in noun/verb analysis is to collect as much relevant information as possible.
- Suitable sources of information are
    - the requirements model;
    - the use case model;
    - the project glossary;
    - anything else (architecture, vision documents, etc.).
- After collecting the documentation, analyze it in a very simple way by highlighting the following:
    - nouns - for example, flight;
    - noun phrases - for example, flight number;
    - verbs - for example, allocate;
    - verb phrases - for example, verify credit card.

b. By using CRC analysis

- CRC Stands for class, responsibilities, and collaborators.
- It is a brainstorming technique.
- It captures on sticky notes the important things in the problem domain.

Class name: BankAccount	
Responsibilities: Maintain balance	Collaborators: Bank

CRC analysis procedure:

- It always be used in conjunction with noun/verb analysis
- It is a <u>two-phase activity.</u>

<u>Phase 1: Brainstorm - gather the information</u>

- The participants are
    - OO analysts,
    - stakeholders, and
    - Domain experts.
    - A facilitator
- The procedure is as follows.
    - **Explain that this is a true brainstorm:**
        - All ideas are accepted as good ideas.
        - Ideas are recorded but *not* debated - never argue about something.
        - Just write it down and then move on.
        - Everything will be analyzed later.

        - **Ask the team members to name the "things**" that operate in their business domain - for example, customer, product:
        - Write each thing on a sticky note - it is a candidate class, or attribute of a class.
        - Stick the note on a wall or whiteboard.
    - **Ask the team to state responsibilities** that those things might have record these in the responsibilities compartment of the note.
    - **Working with the team**, try to identify classes that might work together:
        - To identify classes
        - Rearrange the them
        - draw lines between them

<u>Phase 2: Analyze information</u>

- The participants are OO analysts and domain experts.
- *<u>How do you decide which sticky notes should ·become classes and which should become attributes?</u>*
- Analysis classes *must* represent a crisp abstraction in the problem domain.
- If a note logically seems to be a *part* of another note, this is a good indication that it represents an attribute.
- If a note doesn't seem to be particularly important or has very little interesting behavior, see if it can be made an attribute of another class.
- If in doubt about a note just make it a class.
  <u>We can always refine the model later</u>

c. Finding classes by using the RUP stereotypes:

- According to RUP, it helps to identify three distinct types of analysis class during analysis activity.
    1. «boundary»,
    2. «control», and
    3. «entity» classes.
- A useful technique comes from RUP in the form of RUP stereotypes.
- It is an optional technique.
- Use it as a complement the core noun/verb and CRC card analysis techniques.

1.Finding* «boundary» *classes

a. It exist on the boundary of the <u>system</u> and communicate with <u>external actors.</u>
b. According to RUP there are three types of «boundary» class – <u>Next slide</u>
c. Each communication between an actor and a use case in the model must be enabled by some object in a system.
d. These objects are instances of boundary classes.

Stereotype	Icon	Semantics
«boundary»		a class that mediates interaction between the system and its environment
«control»		a class that encapsulates use-case-specific behavior
«entity»		a class that is used to model persistent information about something

Here the classes are classified as follows:

- **user interface classes –**
    - classes that interface between the system and humans;
- **system interface classes –**
    - classes that interface with other systems

- **Device interface classes –**
    - classes that interface with external devices such as sensors.

Boundary class is indicated by considering what the actor represents. Analysis classes only have key attributes and very high-level responsibilities. «boundary» class represents a GUI.

Actor	Indicates
Represents a human	User interface class
Represents a system	System interface class
Represents a device	Device interface class

-

2. ***Finding* «control» *classes***
    a. These classes are controllers.
    b. It coordinate system behavior that corresponds to one or more use cases.
    c. Find control classes by considering the behavior of the system (Use cases).
    d. If a controller class has a very complicated behavior then
    e. break it down into two or more simpler controllers.

Case study:

System: Designing a course registration
Control class: CourseRegistrationController - that coordinates the whole process.
such a class has a complex behaviour
The CourseRegistrationController might be decomposed into
Registrar, CourseManager, and PersonnelManager classes

3. ***Finding «entity» classes***
    - These classes model information about something.
    - It has simple behavior that getting and setting values.
    - Classes that represent persistent information. Ex: addresses, People (Also called Entity classes)
    - Entity classes Characteristics are as follows
        i. Cut across many use cases;
        ii. are manipulated by control classes;
        iii. provide information to, and accept information from, boundary classes;
        iv. represent key things managed by the system (e.g., Customer);
        v. are often persistent.
    - Entity classes express the logical data structure of the system

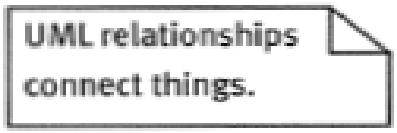
d. Finding classes from other sources
- Like others it is also a worthy.
- Physical objects such as Ex: aircraft, people, and hotels may all indicate classes.
- Paperwork is another rich source of classes. Ex: invoices, orders, and bankbooks
- Known interfaces to the outside world Ex: screens, keyboards, peripherals, and other systems
- Conceptual entities - crucial to the operation of the business. Ex: reward card

e. Relationship
- Relationships are semantic (meaningful) connections between modeling element.
- Few types of relationships

i. between actors and use cases (association);
ii. between use cases and use cases (generalization, «include», «extend»);
iii. between actors and actors (generalization).

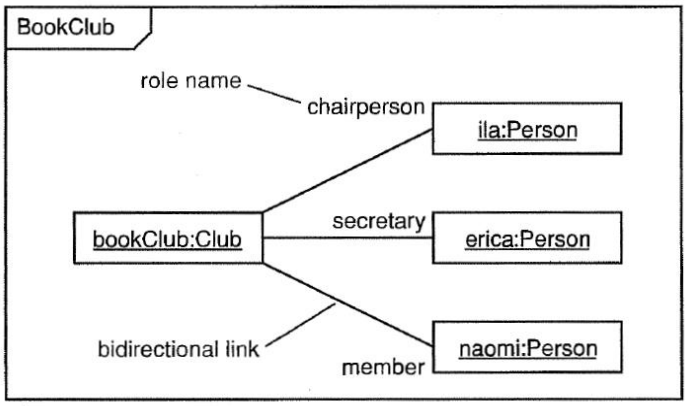
Link:

- A link is a semantic connection between two objects.
- It allows messages to be sent from one object to the other



f. Object Diagrams

- It shows objects and their relationships.
- It derived from the class diagrams. So it depends on class diagram.
- Object diagram is an instance of a class daigram

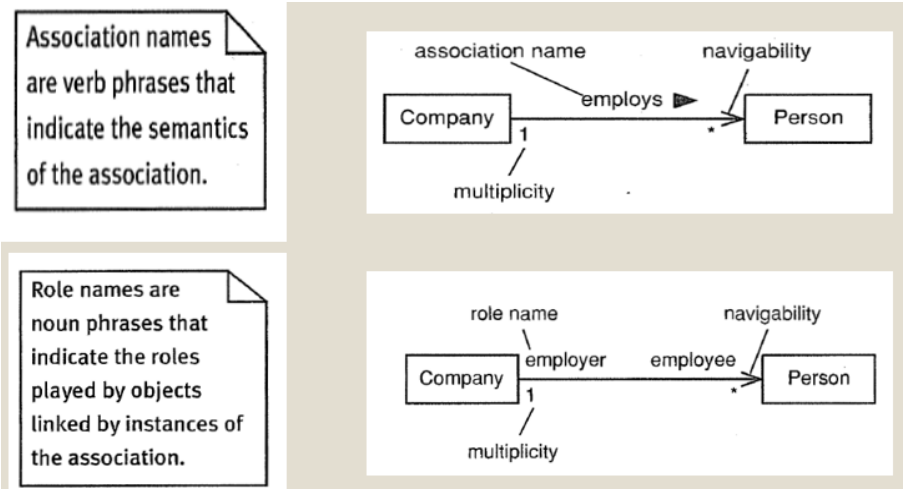


UML 2 specification allows three different modeling idioms

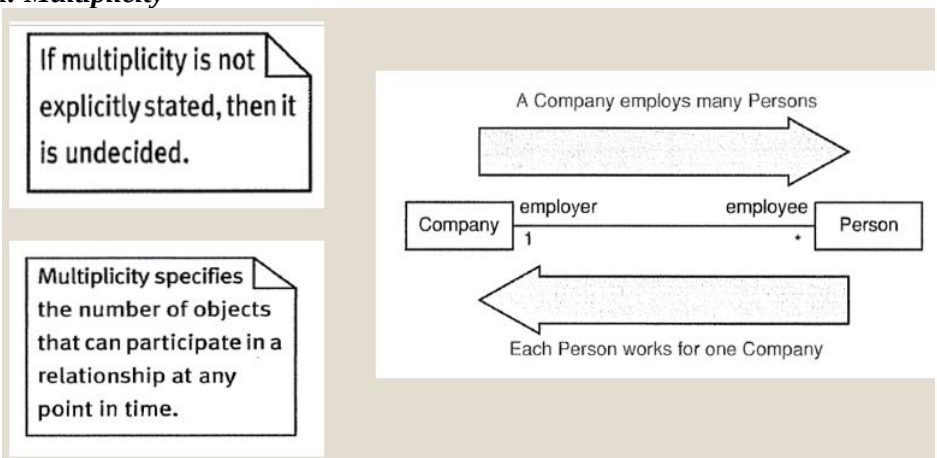
a. all crosses are suppressed;
b. bidirectional associations have no arrows;
c. unidirectional associations have a single arrow.

g. Associations:

- Associations are relationships between classes.
- Links connect objects.
- Associations connect classes.
- Objects are instances of classes,
- Links are instances of associations.



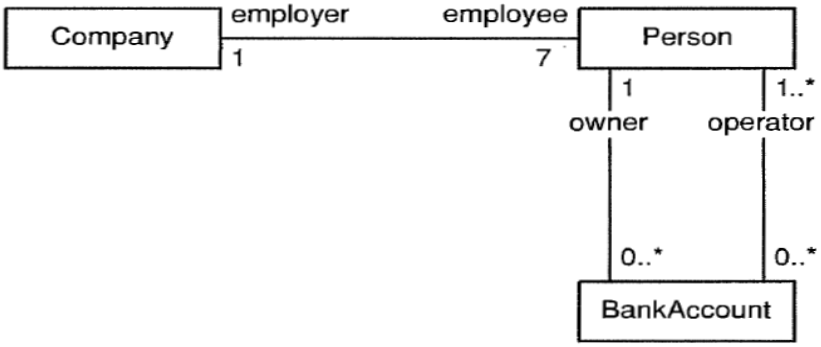
Association: Multiplicity



Adornment	Semantics
0..1	Zero or 1
1	Exactly 1
0..*	Zero or more
*	Zero or more
1..*	1 or more
1..6	1 to 6
1..3, 7..10, 15, 19..*	1 to 3 *or* 7 to 10 *or* 15 exactly *or* 19 to many

Case Study : Multiplicity:

- a Company can have exactly seven employees;
- a Person can be employed by exactly one Company (i.e., in this model a Person can't have more than one job at a time);
- a BankAccount can have exactly one owner;
- a BankAccount can have one or many operators;
- a Person may have zero to many BankAccounts;
- a Person may operate zero to many BankAccounts.

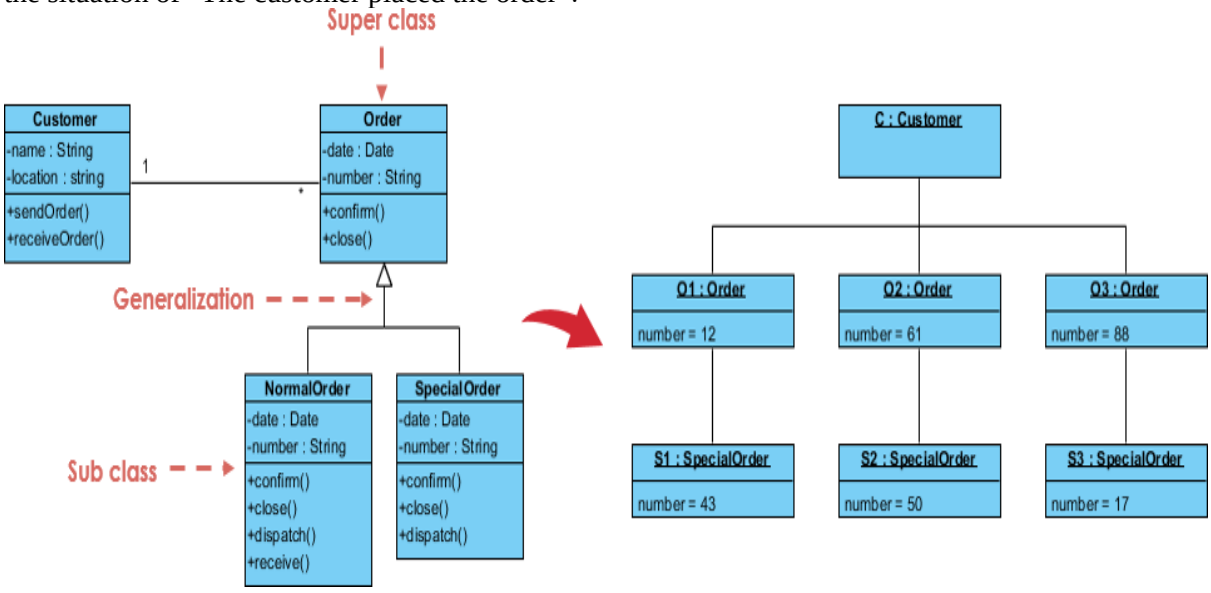


Note:

a. There to be a link between two objects, there must be an association between the classes of those objects.
b. Links depend on associations

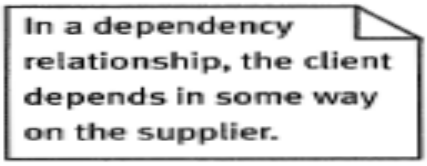
Example: Class Diagram to Object Diagram
Case Study:

Let us consider the following situation and draw an object diagram from the class diagram to the situation of "The customer placed the order".



Dependencies:

- It indicates a relationship between two or more model elements.
- Use dependencies to model relationships between classifiers where one classifier depends on the other.



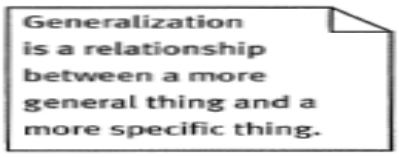
- UML 2 specifies three basic types of dependency,

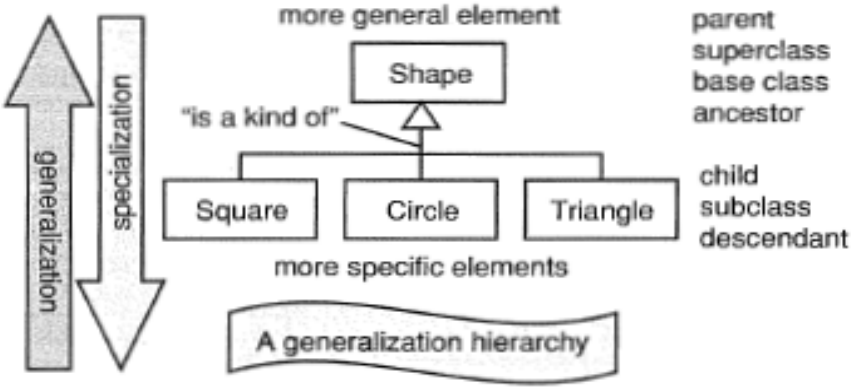
Type	Semantics
Usage	The client uses some of the services made available by the supplier to implement its own behavior – this is the most commonly used type of dependency
Abstraction	This indicates a relationship between client and supplier, where the supplier is more abstract than the client. What do we mean by "more abstract"? This could mean that the supplier is at a different point in development than the client (e.g., in the analysis model rather than the design model)
Permission	The supplier grants some sort of permission for the client to access its contents – this is a way for the supplier to control and limit access to its contents

Type of Dependencies	Stereotypes or Types
Usage	◦ «use»:client makes use of the supplier ◦ «call»: It is between operations ◦ «parameter»:The supplier is a parameter of the client operation ◦ «send»:The client is an operation that sends the supplier ◦ «instantiate»The client is an instance of the supplier.
Abstraction	◦ «trace»: supplier and the client represent the same concept but are in different models. ◦ «substitute»: the client may be substituted for the supplier at runtime. ◦ «refine» : used between elements in the same model. ◦ «derive»: thing can be derived in some way from some other thing.
Permission	◦ «access»: It is between packages. ◦ «import»: similar to «access» ◦ «permit»: Visibility based accesing

Generalization – Class Generalization

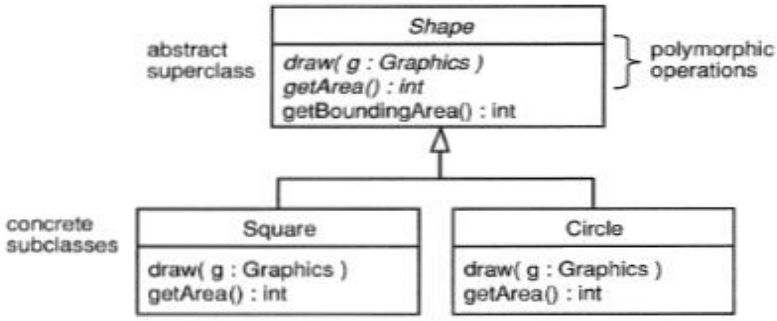
- Generalization applies to all classifiers. Generalization applied to
  - use cases and
  - actors, and
  - classes



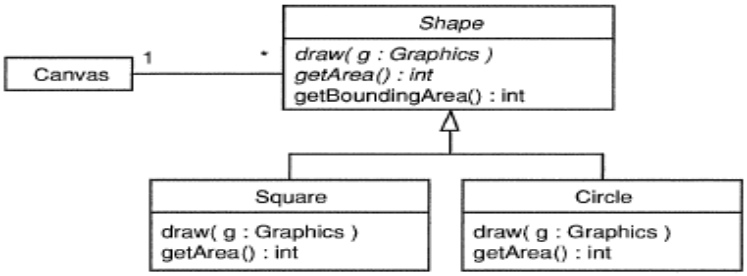


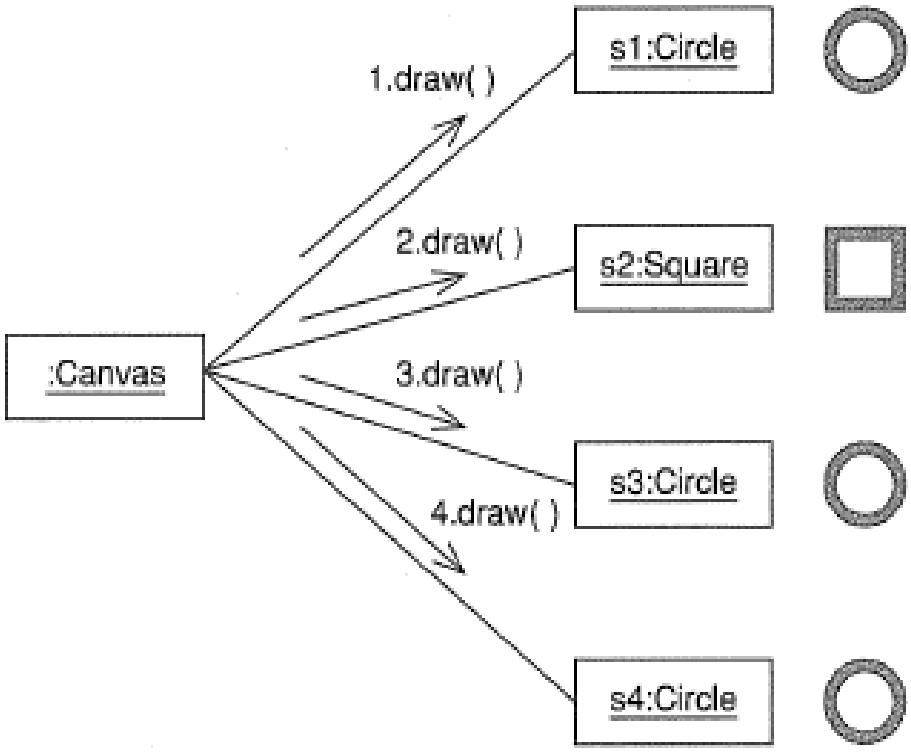
Polymorphism:

- Polymorphism means "many forms.
- Polymorphic operations have many implementations.
- Example:
  - Shape::draw() and
  - Shape::getArea().
- Encapsulation, inheritance, and polymorphism are the "three pillars" of OO.



- Let Canvas is a class.




:Canvas
1.draw( )
2.draw( )
3.draw( )
4.draw( )
s1:Circle
s2:Square
s3:Circle
s4:Circle